PLAN DE PROJET — SOC (Security Operations Center)

Labo de cybersécurité défensive : détection, réponse automatisée et threat intelligence, de A à Z

Rapport de suivi de projet — étape par étape
Environnement : 4 machines virtuelles (2x Ubuntu, 1x Windows Server, 1x Kali Linux)

Composant	Rôle
Kali Linux	Simulation d'attaques (Red Team)
Wazuh	SIEM — collecte de logs, détection, corrélation
Suricata	IDS réseau — détection d'intrusion sur le trafic
Wireshark	Analyse fine du trafic capturé (paquets)
TheHive	Gestion des cas et des incidents (SIRP)
Cortex	Analyse automatisée des observables
VirusTotal	Réputation de fichiers / IP / domaines (via Cortex)
MISP	Threat Intelligence — partage d'IoC
Shuffle	SOAR — orchestration et automatisation (playbooks)
Ollama	IA locale — aide à l'analyse et au résumé
Telegram	Notifications et alertes en temps réel

SOMMAIRE

- 1. Objectifs du projet
- 2. Architecture générale du labo
- 3. Répartition des outils par machine virtuelle
- 4. Plan d'adressage réseau
- 5. Étapes du projet (dans l'ordre)
- Phase 0 — Préparation de l'environnement de virtualisation
- Phase 1 — Déploiement des machines cibles (Ubuntu-SRV1, Win-SRV-2019)
- Phase 2 — Déploiement de Kali Linux (attaquant)
- Phase 3 — Déploiement du socle Docker (Ubuntu-SRV2)
- Phase 4 — Installation et configuration de Wazuh
- Phase 5 — Installation de Suricata (IDS) + intégration Wazuh
- Phase 6 — Installation de Wireshark (analyse de trafic)
- Phase 7 — Installation et configuration de TheHive
- Phase 8 — Installation et configuration de Cortex + VirusTotal
- Phase 9 — Installation et configuration de MISP
- Phase 10 — Déploiement de Shuffle (SOAR) et création des playbooks
- Phase 11 — Intégration d'Ollama (IA locale)
- Phase 12 — Configuration des notifications Telegram
- Phase 13 — Scénarios de test (attaques simulées)
- Phase 14 — Validation, documentation et retour d'expérience
- 6. Flux de données complet (schéma logique)
- 7. Bonnes pratiques et pièges à éviter
- 8. Glossaire

1. OBJECTIFS DU PROJET

Construire, seul et de A à Z, un mini-SOC (Security Operations Center) fonctionnel dans un environnement de laboratoire, afin de comprendre concrètement comment une alerte de sécurité naît, est traitée, enrichie et notifiée — depuis le log brut jusqu'à la décision actionnable par un analyste.

- Simuler des attaques réelles (DoS, scan réseau, IP malveillante) dans un environnement isolé et maîtrisé.
- Détecter ces attaques automatiquement grâce à un SIEM (Wazuh) et un IDS réseau (Suricata).
- Centraliser et trier les incidents dans une plateforme de gestion de cas (TheHive).
- Automatiser l'enrichissement des alertes avec de la threat intelligence (Cortex, VirusTotal, MISP).
- Orchestrer des réponses automatiques via un SOAR (Shuffle).
- Recevoir des notifications en temps réel (Telegram) et bénéficier d'une aide à l'analyse via une IA locale (Ollama).
- Documenter chaque étape pour pouvoir reproduire, expliquer et faire évoluer le labo.

2. ARCHITECTURE GÉNÉRALE DU LABO

L'architecture repose sur 4 machines virtuelles isolées dans un même réseau privé (host-only / internal network), afin qu'aucune attaque ne puisse sortir du labo vers l'extérieur.

Logique de circulation de l'information

- **Kali Linux** attaque les cibles (Ubuntu-SRV1 et Win-SRV-2019).
- **Suricata** (sur Ubuntu-SRV1) inspecte le trafic réseau en direct et génère des alertes IDS.
- **Wazuh** (agents sur les cibles → manager sur Ubuntu-SRV2) collecte les logs systèmes et les alertes Suricata, applique ses règles de corrélation.
- Une alerte Wazuh critique déclenche automatiquement la création d'un cas dans **TheHive**.
- **Cortex** lance ses analyseurs automatiques (dont **VirusTotal** et AbuseIPDB) sur les observables du cas (IP, hash, domaine).
- **MISP** est interrogé pour enrichir avec des indicateurs de compromission (IoC) déjà connus.
- **Shuffle** orchestre le tout : il peut déclencher automatiquement Cortex, interroger VirusTotal/MISP, créer/mettre à jour le cas TheHive, et envoyer la notification finale.
- **Ollama** (IA locale) peut être sollicité pour résumer un cas ou proposer une première qualification.
- **Telegram** reçoit la notification finale (desktop + mobile).

■ *Toutes les analyses (VirusTotal, MISP) se font sur des observables (IP, hash, domaine) et jamais en exposant le labo lui-même sur Internet. Le réseau d'attaque doit rester isolé du réseau de l'hôte physique.*

3. RÉPARTITION DES OUTILS PAR MACHINE VIRTUELLE

Avec 4 VMs disponibles (2x Ubuntu, 1x Windows Server, 1x Kali), voici la répartition retenue. La deuxième machine Ubuntu sert de socle Docker pour héberger tout l'écosystème SOC.

VM	Rôle	Outils installés
VM1 — KALI-LINUX	Attaquant (Red Team)	Kali Linux, Nmap, Hping3, Metasploit, Hydra, (Wireshark en option pour vérifier son propre trafic sortant)
VM2 — UBUNTU-SRV1	Cible + sonde réseau	Ubuntu Server, service applicatif exposé (web/SSH), Agent Wazuh, **Suricata** (IDS), **Wireshark** / tcpdump (capture et analyse de paquets)

VM	Rôle	Outils installés
VM3 — WIN-SRV-2019	Cible Windows	Windows Server 2019, Agent Wazuh, Sysmon (journalisation avancée), pare-feu Windows activé
VM4 — UBUNTU-SRV2	Socle SOC (Docker Host)	Docker + Docker Compose, Wazuh Manager/Indexer/Dashboard, TheHive, Cortex (analyseur **VirusTotal**, AbuseIPDB), MISP, **Shuffle** (SOAR), Ollama (IA locale), Bot Telegram

■ *Pourquoi ce découpage ? Suricata et Wireshark doivent être au plus près du trafic à inspecter : on les place donc sur la cible Ubuntu-SRV1 plutôt que sur le socle Docker. À l'inverse, TheHive, Cortex, MISP et Shuffle sont des services d'orchestration/analyse qui n'ont pas besoin d'être sur la cible : ils tournent ensemble sur le socle Docker (Ubuntu-SRV2) pour simplifier leur mise en réseau.*

4. PLAN D'ADRESSAGE RÉSEAU (exemple)

À adapter selon ton hyperviseur (VirtualBox / VMware / Proxmox). Utiliser un réseau interne dédié, non routé vers Internet pour la partie attaque.

Machine	Adresse IP (exemple)	Réseau
KALI-LINUX	192.168.56.10	Réseau interne labo
UBUNTU-SRV1 (cible + sonde)	192.168.56.11	Réseau interne labo
WIN-SRV-2019 (cible)	192.168.56.12	Réseau interne labo
UBUNTU-SRV2 (socle SOC)	192.168.56.13	Réseau interne + NAT (pour VirusTotal/MISP updates)

Service	Port
Wazuh Dashboard	443
Wazuh Manager (agents)	1514 / 1515
TheHive	9000
Cortex	9001
MISP	8080 / 443
Shuffle	3001
Ollama	11434

5. ÉTAPES DU PROJET — DANS L'ORDRE

PHASE 0 — Préparation de l'environnement de virtualisation

Objectif : Mettre en place l'hyperviseur et le réseau isolé qui accueillera les 4 VMs.

- Installer un hyperviseur (VirtualBox, VMware Workstation, ou Proxmox si tu as un serveur dédié).
- Créer un réseau interne / host-only isolé (ex : 192.168.56.0/24) sans accès direct à ton réseau personnel.
- Prévoir un second adaptateur NAT sur UBUNTU-SRV2 uniquement, pour permettre les mises à jour MISP / téléchargement des images Docker / requêtes VirusTotal.
- Allouer les ressources : minimum 8 Go RAM et 4 vCPU pour UBUNTU-SRV2 (elle héberge le plus de conteneurs), 2 Go RAM pour les autres VMs.

Critère de validation : Les 4 VMs se pinguent entre elles sur le réseau interne, et seule UBUNTU-SRV2 a accès à Internet.

PHASE 1 — Déploiement des machines cibles (UBUNTU-SRV1, WIN-SRV-2019)

Objectif : Avoir deux cibles réalistes à surveiller, représentatives d'une petite infrastructure d'entreprise.

- Installer Ubuntu Server (LTS) sur UBUNTU-SRV1, avec un service exposé (ex : Apache/Nginx ou SSH) pour donner une surface d'attaque réaliste.
- Installer Windows Server 2019 sur WIN-SRV-2019, activer la journalisation d'événements de sécurité.
- Installer Sysmon sur WIN-SRV-2019 pour une journalisation plus fine (process, connexions réseau).
- Configurer une adresse IP statique sur chaque VM.

Critère de validation : Les deux cibles sont accessibles depuis Kali (ping + un service ouvert détectable par Nmap).

PHASE 2 — Déploiement de Kali Linux (attaquant)

Objectif : Disposer d'une plateforme d'attaque avec les outils nécessaires aux scénarios de test.

- Installer/mettre à jour Kali Linux (`apt update && apt full-upgrade`).
- Vérifier la présence des outils : Nmap, Hping3, Metasploit, Hydra.
- Confirmer la connectivité réseau vers UBUNTU-SRV1 et WIN-SRV-2019 uniquement (pas d'accès direct à UBUNTU-SRV2 pour l'instant, afin de ne pas fausser les tests).

Critère de validation : `nmap -sV 192.168.56.11` renvoie bien les services ouverts de la cible.

PHASE 3 — Déploiement du socle Docker (UBUNTU-SRV2)

Objectif : Préparer la machine qui hébergera tout l'écosystème SOC (Wazuh, TheHive, Cortex, MISP, Shuffle, Ollama).

- Installer Docker Engine et Docker Compose.
- Créer une arborescence de projet claire, un dossier par outil (ex : ~/soc-lab/wazuh, ~/soc-lab/thehive, ~/soc-lab/misp, ~/soc-lab/shuffle).
- Créer un réseau Docker dédié (`docker network create soc-net`) pour que les conteneurs communiquent entre eux par leur nom.
- Vérifier les ressources disponibles (RAM/CPU) avant de lancer plusieurs stacks lourdes en parallèle.

```
sudo apt update && sudo apt install -y docker.io docker-compose-plugin
```

```
sudo usermod -aG docker $USER

docker network create soc-net
```

Critère de validation : `docker run hello-world` fonctionne, et `docker network ls` liste bien soc-net.

PHASE 4 — Installation et configuration de Wazuh

Objectif : Mettre en place le SIEM central : manager + indexer + dashboard sur UBUNTU-SRV2, agents sur les cibles.

- Déployer la stack Wazuh (manager, indexer, dashboard) via le docker-compose officiel sur UBUNTU-SRV2.
- Installer l'agent Wazuh sur UBUNTU-SRV1 et le faire pointer vers le manager.
- Installer l'agent Wazuh sur WIN-SRV-2019 et le faire pointer vers le manager.
- Créer/activer des règles de détection pour : scan de ports (Nmap), flood ICMP (DoS), tentatives de brute force.
- Configurer l'intégration Wazuh → TheHive (module d'intégration natif ou script personnalisé) pour créer un cas automatiquement sur alerte critique.

Critère de validation : Le dashboard Wazuh affiche les deux agents en statut 'Active', et une alerte de test (ping simple) remonte bien dans l'interface.

PHASE 5 — Installation de Suricata (IDS) + intégration Wazuh

Objectif : Ajouter une couche de détection réseau complémentaire à Wazuh, au plus près du trafic.

- Installer Suricata directement sur UBUNTU-SRV1 (en mode IDS, sur l'interface réseau qui reçoit le trafic de Kali).
- Activer les règles Emerging Threats (ET Open) : scan, DoS, exploitation connue.
- Configurer Suricata pour écrire ses logs au format eve.json.
- Configurer l'agent Wazuh sur UBUNTU-SRV1 pour lire et faire remonter le fichier eve.json vers le manager Wazuh (decoder/ruleset Suricata pour Wazuh).

```
sudo apt install -y suricata suricata-update

sudo suricata-update

sudo systemctl enable --now suricata
```

Critère de validation : Un scan Nmap depuis Kali génère une entrée dans /var/log/suricata/eve.json ET une alerte visible dans le dashboard Wazuh.

PHASE 6 — Installation de Wireshark (analyse de trafic)

Objectif : Disposer d'un outil d'inspection manuelle fine du trafic, en complément de la détection automatique.

- Installer Wireshark (ou tshark en ligne de commande) sur UBUNTU-SRV1.
- Utiliser tcpdump pour capturer le trafic pendant les scénarios d'attaque, puis ouvrir les captures (.pcap) dans Wireshark pour les analyser en détail.
- Documenter, pour chaque scénario, les indicateurs visibles dans Wireshark (ex : nombre de paquets ICMP/seconde, IP sources randomisées lors du DoS).

```
sudo apt install -y wireshark tshark

sudo tcpdump -i eth0 -w attaque_dos.pcap
```

Critère de validation : Le fichier .pcap capturé pendant une attaque s'ouvre dans Wireshark et permet de retrouver visuellement le pattern de l'attaque.

PHASE 7 — Installation et configuration de TheHive

Objectif : Centraliser la gestion des cas et incidents remontés par Wazuh.

- Déployer TheHive via Docker Compose sur UBUNTU-SRV2 (port 9000).
- Créer les utilisateurs/analystes et les templates de cas (ex : 'Scan réseau', 'DoS', 'IP malveillante').
- Connecter TheHive à Cortex (clé API) pour permettre le lancement d'analyseurs directement depuis un cas.
- Vérifier la réception automatique des cas créés par l'intégration Wazuh → TheHive.

Critère de validation : Une alerte Wazuh critique crée automatiquement un cas visible dans l'interface TheHive, avec les bons observables (IP source, type d'attaque).

PHASE 8 — Installation et configuration de Cortex + VirusTotal

Objectif : Automatiser l'enrichissement des observables (IP, hash, domaine) via des analyseurs externes.

- Déployer Cortex via Docker Compose sur UBUNTU-SRV2 (port 9001), connecté à sa base Elasticsearch.
- Créer un compte VirusTotal (API gratuite pour usage lab) et récupérer la clé API.
- Activer l'analyseur 'VirusTotal_GetReport' dans Cortex avec la clé API.
- Activer également l'analyseur AbuseIPDB pour la réputation d'IP.
- Tester en lançant l'analyse VirusTotal sur une IP connue malveillante depuis un cas TheHive.

Critère de validation : L'analyse VirusTotal renvoie un score de détection exploitable (ex : X/90 moteurs antivirus l'identifient comme malveillante).

PHASE 9 — Installation et configuration de MISP

Objectif : Disposer d'une base de threat intelligence locale, alimentée par des flux publics et par les cas du labo.

- Déployer MISP via Docker Compose sur UBUNTU-SRV2 (port 8080/443).
- Importer quelques flux publics d'IoC (ex : listes d'IP malveillantes connues) pour peupler la base.
- Créer les événements MISP correspondant aux IoC découverts pendant les tests (IP de Kali comme référence pédagogique, IP malveillante réelle analysée via VirusTotal).
- Connecter Cortex à MISP (analyseur MISP) pour interroger automatiquement la base lors de l'analyse d'un observable.

Critère de validation : Une recherche d'IP dans MISP retourne les événements associés, et Cortex peut interroger MISP comme un analyseur classique.

PHASE 10 — Déploiement de Shuffle (SOAR) et création des playbooks

Objectif : Automatiser la chaîne complète (orchestration) : de l'alerte à la notification, sans intervention manuelle.

- Déployer Shuffle via Docker Compose sur UBUNTU-SRV2 (port 3001).
- Créer un premier workflow : réception d'un webhook depuis TheHive (nouveau cas) → extraction de l'IP source.

- Ajouter une étape d'enrichissement automatique : appel à l'API VirusTotal et/ou à Cortex.
- Ajouter une étape de décision (playbook) : si score de réputation élevé → mise à jour du cas TheHive en 'critique' + envoi de la notification Telegram.
- Tester le workflow de bout en bout avec un cas réel généré lors d'un scénario d'attaque.

Critère de validation : Un nouveau cas TheHive déclenche automatiquement le workflow Shuffle, qui enrichit l'observable et notifie sans action manuelle.

PHASE 11 — Intégration d'Ollama (IA locale)

Objectif : Ajouter une aide à l'analyse (résumé de cas, proposition de qualification) sans dépendre du cloud.

- Installer Ollama sur UBUNTU-SRV2 et télécharger un modèle léger adapté (ex : llama3 8B ou mistral 7B selon les ressources disponibles).
- Exposer l'API locale d'Ollama (port 11434) au réseau Docker interne.
- Créer une étape Shuffle qui envoie le résumé du cas TheHive (description + observables) à Ollama, et récupère une synthèse en langage naturel.
- Ajouter cette synthèse comme commentaire automatique dans le cas TheHive.

Critère de validation : Un cas TheHive reçoit automatiquement un résumé généré par Ollama, cohérent avec les données de l'incident.

PHASE 12 — Configuration des notifications Telegram

Objectif : Être notifié en temps réel, sur desktop et mobile, dès qu'un incident est qualifié.

- Créer un bot Telegram via @BotFather et récupérer le token.
- Créer un canal ou groupe dédié aux alertes SOC, et y ajouter le bot.
- Ajouter une étape finale dans le workflow Shuffle qui envoie un message formaté (type d'attaque, IP source, score de réputation, lien vers le cas TheHive) sur Telegram.
- Vérifier la réception sur Telegram Desktop et Telegram Mobile.

Critère de validation : Une alerte critique de bout en bout arrive sur Telegram en moins de quelques secondes après l'attaque.

PHASE 13 — Scénarios de test (attaques simulées)

Objectif : Valider l'ensemble de la chaîne avec des scénarios réalistes et répétables.

- **Scénario A — DoS :** depuis Kali, `hping3 --icmp --rand-source --flood ` vers UBUNTU-SRV1. Vérifier la détection Suricata + Wazuh + création du cas + notification.
- **Scénario B — Scan réseau :** `nmap -sS -p- ` depuis Kali. Vérifier la remontée du scan avec IP source, ports ciblés et règle déclenchée.
- **Scénario C — IP malveillante connue :** soumettre une IP flaggée publiquement à Cortex/VirusTotal/AbuseIPDB et vérifier l'enrichissement (score d'abus, catégories d'attaque).
- **Scénario D — Brute force :** `hydra` sur le service SSH de UBUNTU-SRV1, vérifier la détection des tentatives répétées.
- Pour chaque scénario, capturer le trafic avec Wireshark/tcpdump et conserver la capture comme preuve pédagogique.

Critère de validation : Chaque scénario est détecté, catalogué, enrichi et notifié sans intervention manuelle, et documenté avec captures d'écran + pcap.

PHASE 14 — Validation, documentation et retour d'expérience

Objectif : Consolider le projet pour qu'il soit compréhensible, reproductible et présentable.

- Rédiger une fiche par scénario (objectif, commande utilisée, détection observée, capture d'écran).
- Faire un schéma d'architecture final à jour (avec Shuffle, VirusTotal, Suricata, Wireshark ajoutés).
- Lister les limites du labo (faux positifs éventuels, ressources limitées, pas de haute disponibilité).
- Identifier les axes d'amélioration (ex : ajout d'un reverse proxy, durcissement des VMs cibles, ajout de nouveaux playbooks Shuffle).

Critère de validation : Le projet est documenté de façon à être compris et rejoué par une autre personne sans explication orale.

6. FLUX DE DONNÉES COMPLET (SCHÉMA LOGIQUE)

- 1. Kali Linux lance une attaque (DoS, scan, brute force) vers UBUNTU-SRV1 ou WIN-SRV-2019.
- 2. Suricata (sur UBUNTU-SRV1) inspecte le trafic réseau en direct et journalise une alerte IDS (eve.json).
- 3. L'agent Wazuh (sur la cible) collecte les logs système + les alertes Suricata, et les envoie au Wazuh Manager (UBUNTU-SRV2).
- 4. Le Wazuh Manager applique ses règles de corrélation et génère une alerte qualifiée (niveau de sévérité).
- 5. Si l'alerte est critique, un cas est automatiquement créé dans TheHive, avec les observables extraits (IP source, ports, type d'attaque).
- 6. Shuffle détecte le nouveau cas (webhook) et déclenche le playbook d'enrichissement.
- 7. Cortex (via Shuffle ou manuellement) interroge VirusTotal et AbuseIPDB sur l'observable (IP).
- 8. MISP est interrogé pour vérifier si l'IoC est déjà connu dans la base de threat intelligence.
- 9. Ollama génère un résumé en langage naturel du cas, ajouté comme commentaire dans TheHive.
- 10. Shuffle met à jour la sévérité du cas et envoie la notification finale sur Telegram (desktop + mobile).

Temps total observé dans le projet original : détection en quelques secondes, zéro intervention manuelle entre l'attaque et la notification.

7. BONNES PRATIQUES ET PIÈGES À ÉVITER

- **Isolation réseau :** ne jamais exposer Kali Linux ou les cibles vulnérables directement sur Internet.
- **Snapshots :** prendre un snapshot de chaque VM après une installation stable, avant chaque test destructif.
- **Clés API :** ne jamais committer les clés VirusTotal/MISP/Telegram dans un dépôt Git public ; utiliser des variables d'environnement (.env).
- **Ressources :** Wazuh + TheHive + Cortex + MISP + Shuffle + Ollama en simultané demandent beaucoup de RAM — surveiller la consommation et désactiver les stacks non utilisées si besoin.
- **Faux positifs :** ajuster progressivement les règles Wazuh/Suricata pour éviter le bruit qui noierait une vraie alerte.
- **Traçabilité :** horodater et documenter chaque test pour pouvoir comparer l'attendu et l'observé.
- **Ordre d'installation :** toujours valider une brique (ex : Wazuh seul) avant d'ajouter la suivante (ex : intégration TheHive), pour isoler facilement les problèmes.

8. GLOSSAIRE

Terme	Définition
SOC	Security Operations Center — équipe/dispositif de surveillance continue de la sécurité.
SIEM	Security Information and Event Management — collecte et corrélation de logs (Wazuh).
IDS	Intrusion Detection System — détection d'intrusion sur le trafic réseau (Suricata).
SIRP	Security Incident Response Platform — gestion des cas et incidents (TheHive).
SOAR	Security Orchestration, Automation and Response — automatisation des réponses (Shuffle).
IoC	Indicator of Compromise — indicateur de compromission (IP, hash, domaine malveillant).

Terme	Définition
Threat Intelligence	Renseignement sur les menaces connues, partagé/consulté via MISP.
Observable	Donnée technique extraite d'un incident (IP, hash, URL...) analysable par Cortex.
Playbook	Enchaînement automatisé d'actions défini dans un SOAR (Shuffle).
DoS / DDoS	Attaque par déni de service, visant à saturer un service (disponibilité).